

MEGAJAW

Autonomous Bottle Recovery Robot



Project Documentation

Github: <https://github.com/zeyad-shaban/megatroll>

Team Members

Name	Student ID
Ahmed Tamer Mohamed	221005802
Hazem Mohamed Ahmed	221006745
Ahmed Mohamed Shawky	221004411
Zeyad Youssef Shaban	221004741

1. Project Overview

MegaJaw is a ROS 2 Jazzy autonomous mobile robot built for plastic bottle recovery in public and semi-enclosed spaces. It integrates a YOLO-based perception stack, proportional motion control, a two-finger gripper, and a browser-based control dashboard all running on a Raspberry Pi.

The same autonomy code runs identically in Gazebo simulation and on physical hardware. The only change at deployment time is the backend_driver Xacro parameter, which swaps the hardware abstraction layer without touching the autonomy or perception layers.

1.1 Key Capabilities

- Monocular depth estimation — no depth camera required (pinhole camera formula)
- Three interchangeable hardware backends: Gazebo simulation, STM32 UART, Raspberry Pi GPIO
- YOLO NCNN on-device inference optimised for Raspberry Pi
- Five-state FSM: IDLE → TO_TARGET → GRIPPER_CLOSE → GO_HOME → GRIPPER_OPEN
- Browser-based teleoperation dashboard with live video, joystick, and autonomous toggle
- ROSBridge WebSocket integration for seamless ROS ↔ browser communication



2. Problem Statement

Plastic bottle litter in public and semi-enclosed spaces is a persistent environmental problem. Manual collection is tedious and does not scale. Autonomous ground robots offer a more scalable alternative, but they demand tight integration of perception, motion control, and manipulation in a compact, low-cost platform.

MegaJaw specifically targets the full closed-loop recovery cycle: detect a target, navigate toward it with enough precision to engage the gripper, physically capture the object, and return to a drop-off point — all without human intervention. This requires:

- Reliable monocular depth estimation (no depth camera available)
- Stable target tracking across frames to prevent mid-approach switching
- A deterministic state machine that sequences approach, grasp, retreat, and release
- A robust hardware abstraction layer that runs the same code in simulation and on real hardware

3. Proposed Solution

MegaJaw addresses the bottle recovery problem through four tightly integrated subsystems:

3.1 Differential-Drive Base

A differential-drive base handles planar motion through the standard `ros2_control diff_drive_controller`, which accepts `TwistStamped` velocity commands and translates them to per-wheel commands for whichever hardware backend is active. The autonomy layer never needs to know what hardware is underneath.

3.2 YOLO Perception Stack

Perception runs an Ultralytics YOLO model exported to NCNN format on-device. Separate models handle simulation (detecting Gazebo red-box stand-ins at 256×256) and real hardware (detecting physical bottles from a phone camera stream at 320×320). Monocular depth uses the pinhole camera formula:

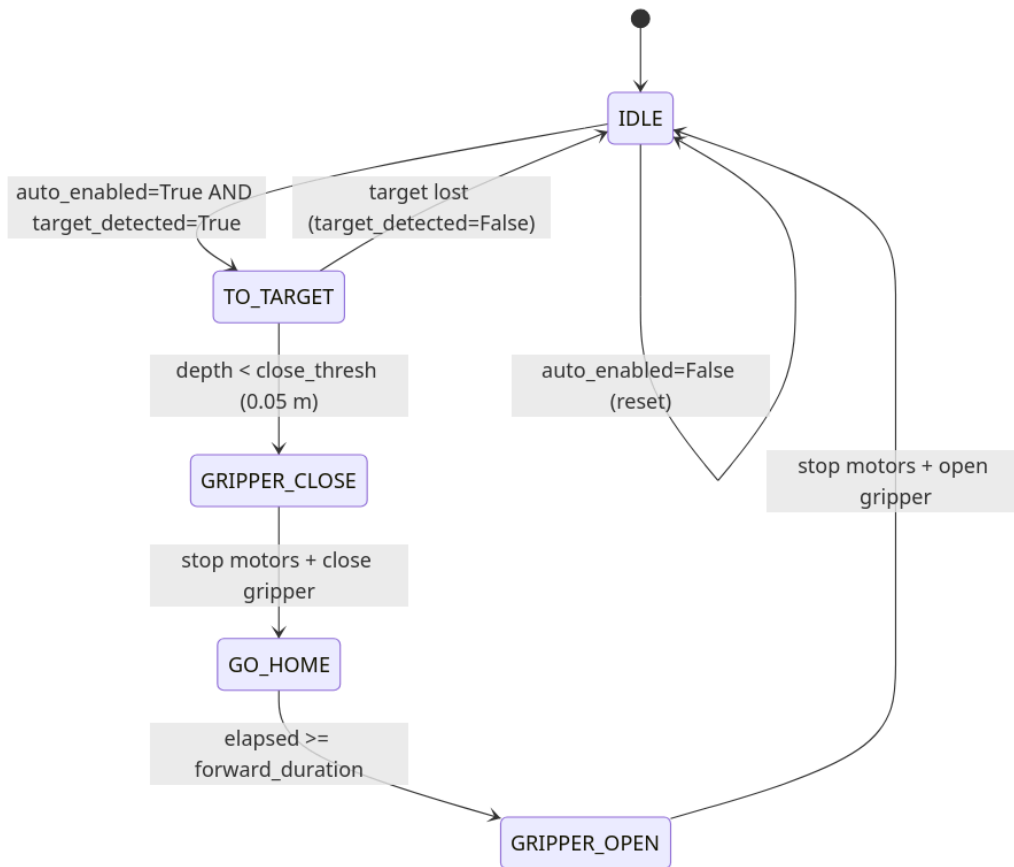
```
z = (W × fx) / p

where:
W  = known physical object width (metres)
fx = focal length (pixels)
p  = bounding-box width in pixels
```

No depth sensor is needed. On real hardware, both width and height estimates are averaged for improved accuracy.

3.3 Five-State Autonomy FSM

Autonomy is implemented as a five-state finite state machine (FSM) in `fsm_node.py`. It uses proportional control on horizontal pixel error (`err_x`) for angular velocity and on estimated depth for linear velocity, transitioning states based on depth threshold and elapsed time.

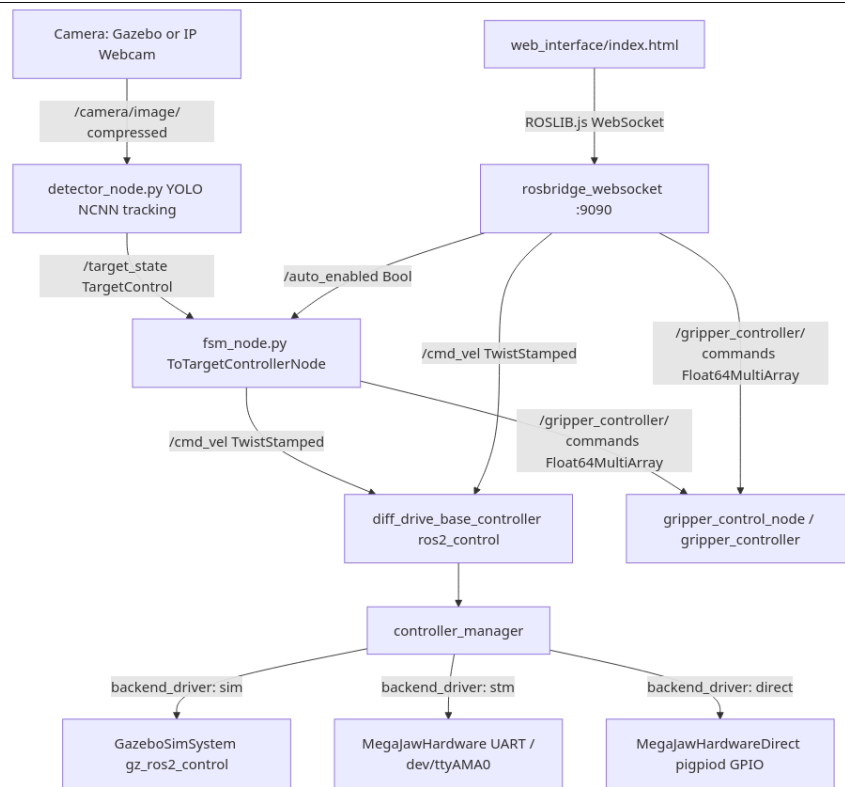


3.4 Hardware Abstraction Layer

The hardware abstraction layer is a `ros2_control` `SystemInterface` plugin with three interchangeable backends selected at launch via the `backend_driver` Xacro argument. This allows the identical workspace to deploy across simulation, STM32-based hardware, and direct Raspberry Pi GPIO configurations without modifying any autonomy or control code.

4. System Architecture

The diagram below shows the complete data flow from the browser dashboard through ROSBridge, the perception and autonomy stack, ros2_control, and the three hardware backends.

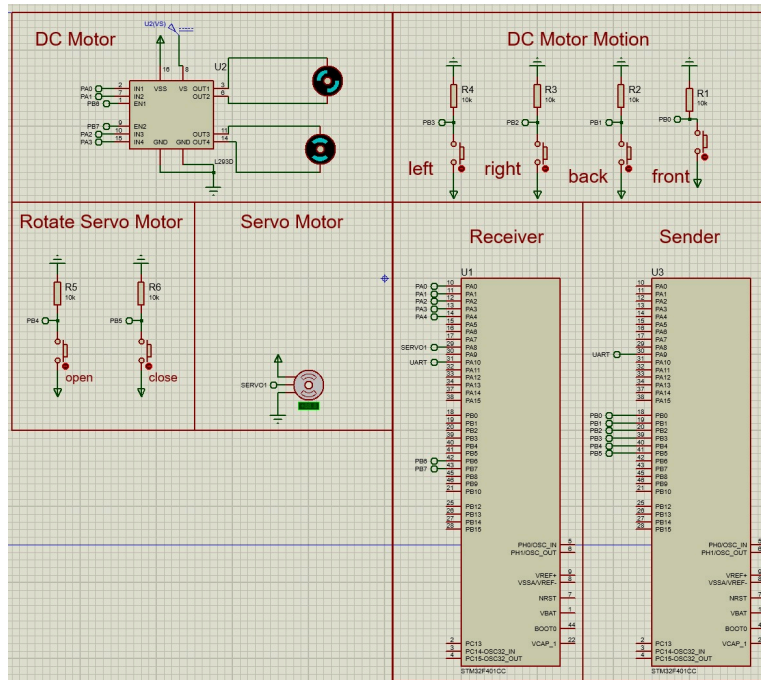


Key architectural decisions:

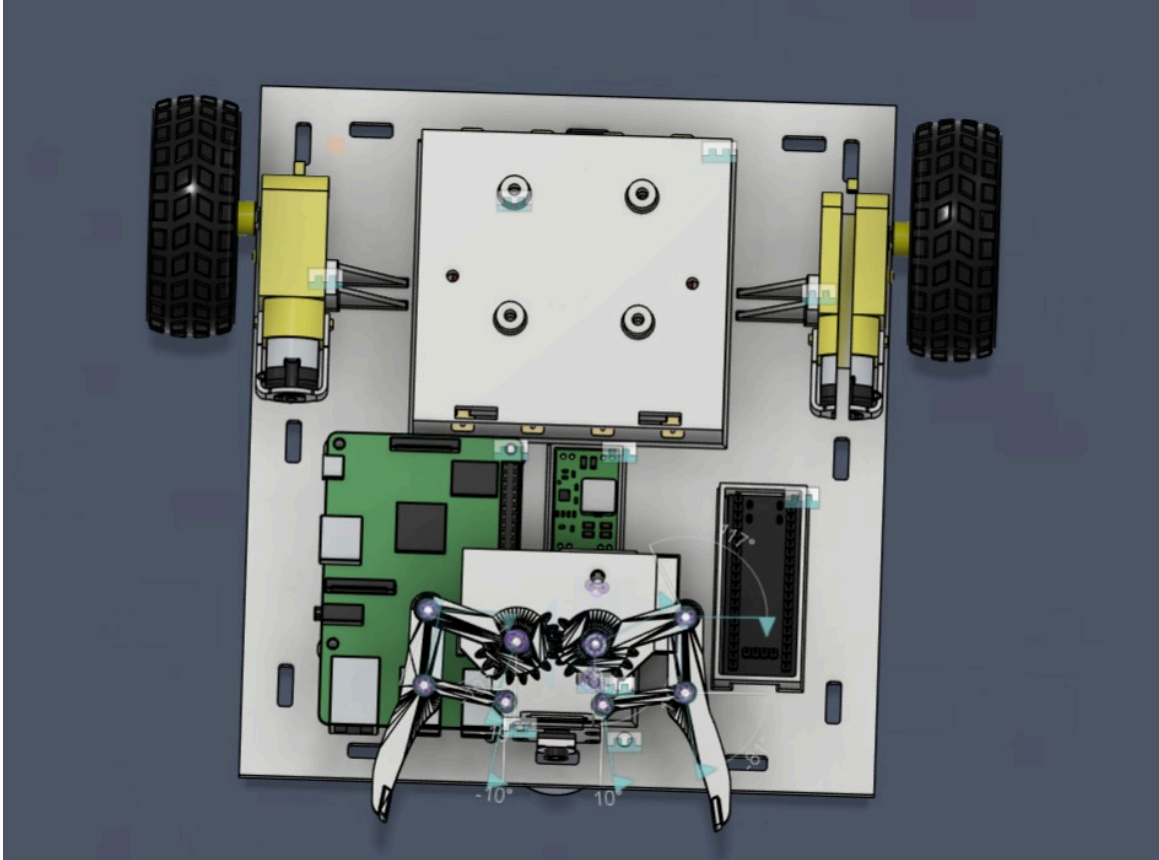
- All intra-process communication uses ROS 2 topics, giving uniform observability via ros2 topic echo regardless of backend.
- The browser talks to the robot exclusively through ROSBridge on port 9090 — no custom server required.
- The single backend_driver Xacro argument at launch time is the only deployment difference between simulation and physical hardware.

5. Hardware

5.1 Component Overview

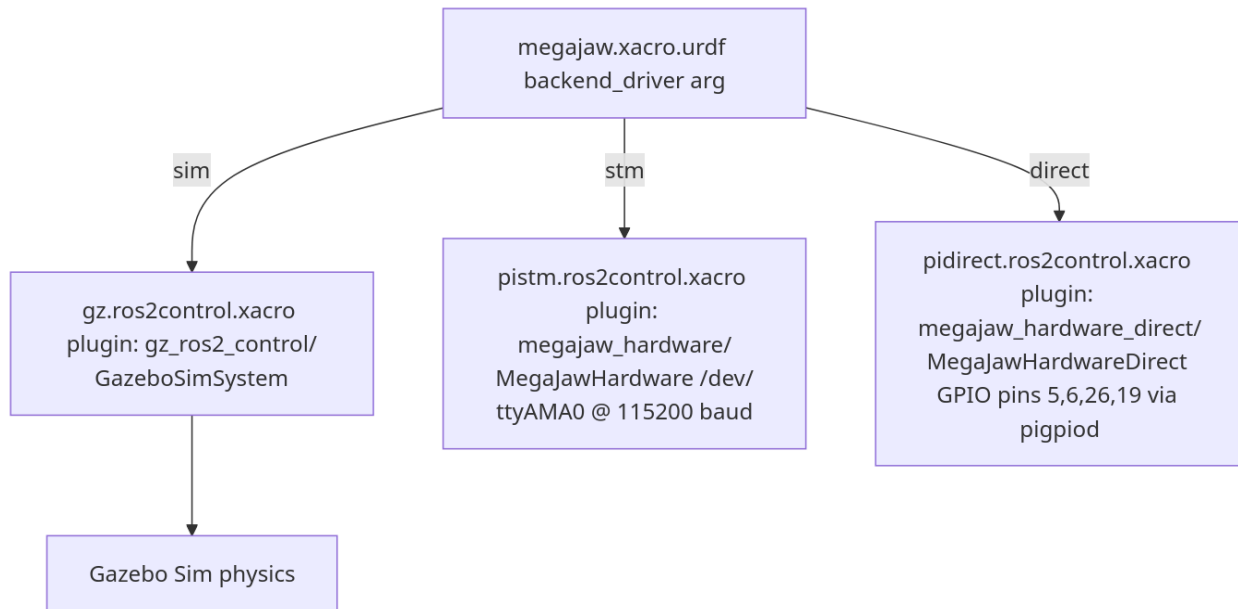


Component	Role
Raspberry Pi (Ubuntu + ROS 2 Jazzy)	Main compute: runs all ROS 2 nodes, ros2_control, ROSBridge, and perception stack
STM32 microcontroller	Low-level motor and gripper actuation over UART; receives 6-byte motor frames and 3-byte gripper frames
Two DC motors	Differential-drive wheel actuation; driven by H-bridge from STM32 or directly via RPi GPIO
H-bridge motor driver	Converts PWM signals from STM32 (or RPi GPIO) to bidirectional motor current
Buck converter / power stage	Steps down battery voltage to logic and motor supply levels
Mobile phone (IP Webcam)	Streams MJPEG video over HTTP; accessed by camera_driver_node
Custom two-finger gripper	Two revolute finger links (0–0.7 rad / -0.7–0 rad) actuated by servo via STM32 or Gazebo
MDF chassis	Custom-fabricated 200 × 180 × 50 mm base with front-mounted gripper and rear caster
Front caster (10 mm radius)	Passive sphere link providing a third contact point for stable differential drive



5.2 Hardware Backends

The backend_driver Xacro argument in megajaw.xacro.urdf selects which ros2_control hardware plugin is compiled into the robot description at launch time. The three conditional blocks are mutually exclusive



backend_driver	Plugin Class	Transport	Use Case
sim	gz_ros2_control/GazeboSimSystem	Gazebo internal	Full simulation with physics and gripper position control
stm	megajaw_hardware/MegaJawHardware	UART /dev/ttyAMA0 115200 baud	Real robot with STM32 handling motor PWM and gripper servo
direct	megajaw_hardware_direct/MegaJawHardwareDirect	RPi GPIO via pigpiod	Real robot without STM32; RPi drives H-bridge pins directly

6. Software Stack

Technology	Version / Package	Role
ROS 2 Jazzy	ros-jazzy-*	Core middleware, topic transport, parameter server, lifecycle management
Gazebo Sim	ros_gz_sim, ros_gz_bridge	Physics simulation, sensor simulation, ROS-Gazebo bridge
ros2_control	controller_manager, diff_drive_controller	Hardware abstraction, differential drive kinematics, gripper position control
Xacro	xacro	Parametric URDF generation; backend_driver selects hardware plugin
ROSBridge	rosbridge_server	WebSocket JSON bridge between browser and ROS 2 topics
Ultralytics YOLO	ultralytics (Python)	YOLO model training, inference, and NCNN export
NCNN	ncnn (Python)	Lightweight neural network inference runtime for on-device detection
OpenCV	cv2	Camera frame capture, undistortion, rotation, and debug visualisation
NumPy	numpy	Array operations for bounding box processing and camera matrix math
PySerial	serial	UART communication in megajaw_uart_bridge
pigpio	pigpio / pigpiod	GPIO PWM control for megajaw_hardware_direct backend
ROSLIB.js	CDN	Browser-side ROS 2 topic pub/sub over WebSocket
Tailwind CSS	tailwind.js	Web dashboard styling
Ubuntu	24.04 Noble	Host OS on Raspberry Pi and development machine

7. Package Structure

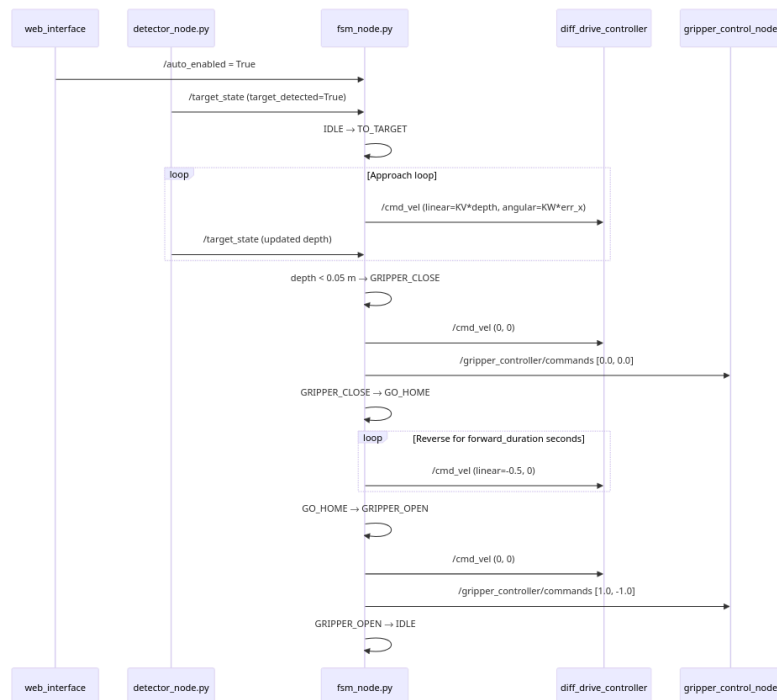
The workspace contains eight ROS 2 packages under src/:

Package	Key Nodes / Files	Purpose
megajaw_bringup	gz.launch.py, hardware.launch.py, display.launch.py	System startup orchestration, controller configs, worlds, RViz
megajaw_description	megajaw.xacro.urdf, body.xacro, *ros2control.xacro	Robot kinematic and sensor model; URDF/Xacro files
megajaw_brain	detector_node.py, fsm_node.py, constants.py	YOLO NCNN perception and five-state FSM autonomy
megajaw_hardware	megajaw_interface.cpp, gripper_control_node.cpp	STM32 UART hardware interface (C++), gripper node, camera node
megajaw_hardware_direct	MegaJawHardwareDirect plugin	Direct Raspberry Pi GPIO motor control without STM32
megajaw_interfaces	TargetControl.msg	Custom ROS 2 message definitions
megajaw_uart_bridge	uart_bridge_node.py	Standalone TwistStamped → UART bridge, bypasses ros2_control
web_interface	index.html, index.js	Browser teleoperation dashboard

8. Autonomy Pipeline

8.1 Pipeline Sequence

The complete autonomy sequence from operator enable to object drop-off:



8.2 Proportional Control

The FSM uses proportional controllers for both motion axes:

- Angular velocity: $w = KW \times \text{err_x}$ (clamped to $\pm W_MAX = 0.7$ rad/s)
- Linear velocity: $v = KV \times \text{depth}$ (clamped to $\pm V_MAX = 0.6$ m/s)

The forward_duration is recorded as the elapsed time from entering TO_TARGET to transitioning to GRIPPER_CLOSE. The GO_HOME state reverses at -0.5 m/s for exactly that duration, giving a time-symmetric return path to the drop-off zone.

8.3 Target Locking

Target locking prevents the FSM from switching targets mid-approach. Once a track ID is locked by detector_node.py, the detector keeps reporting that specific ID's bounding box even if other detections appear. A grace buffer of max_lost_frames (default 30 frames) handles brief occlusions. If the locked ID goes missing beyond the grace buffer, the lock releases and re-acquisition begins by selecting the highest-scoring detection by area $\times$ confidence.

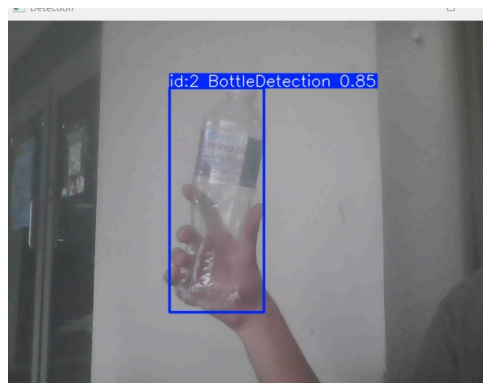
9. Object Detection & Perception

9.1 YOLO Model

DetectorNode subscribes to /camera/image/compressed and calls model.track() using the Ultralytics YOLO API with a persistent tracker configured by tracker_conf.yaml. Two separate NCNN-exported YOLO models are deployed:

- best_ncnn_model_gz — 256 $\times$ 256 input, detects Gazebo red-box stand-ins in simulation
- best_ncnn_model_real — 320 $\times$ 320 input, detects physical water bottles on real hardware

The model is a YOLO26n architecture trained for 200 epochs with heavy augmentation: rotation $\pm 90^\circ$, HSV shifts, mosaic composition, and horizontal/vertical flips.



9.2 Depth Estimation

Depth estimation uses the pinhole camera model. Two configurations are supported:

Simulation Depth

$$fx = \text{GZ_CAM_IMG_WIDTH} / (2 \times \tan(\text{GZ_CAM_HFOV} / 2))$$

```
= 640 / (2 * tan(1.3962634 / 2))

Z = (OBJ_WIDTH_METERS * fx) / bbox_w_px
```

Real Hardware Depth

On real hardware, the node uses calibrated intrinsics and distortion coefficients. The frame is first undistorted with `cv2.undistort()` and rotated 90° clockwise to correct for phone camera orientation. Depth is then estimated from both width and height dimensions and averaged:

```
Calibrated intrinsics:
fx = 1497.70202,  fy = 1435.16210
Distortion: [0.09600973, -1.13969576, 0.00809251, -0.01848817, 1.54268659]

depth_w = (fx * OBJ_WIDTH_METERS) / bbox_w_px      # OBJ_WIDTH_METERS = 0.064 m
depth_h = (fy * OBJ_HEIGHT_METERS) / bbox_h_px      # OBJ_HEIGHT_METERS = 0.250 m
depth    = (depth_w + depth_h) / 2.0
```

10. Communication Protocol ---

10.1 Motor UART Frame (STM32 Backend)

The STM32 motor backend uses a binary UART framing protocol. `MotorDriver::sendToSTM32()` packs a 6-byte little-endian frame at 115200 baud:

```
Frame format:
[0xAA][0x55][left_pwm_low][left_pwm_high][right_pwm_low][right_pwm_high]

0xAA, 0x55    = header magic bytes
left_pwm      = signed 16-bit integer in range [-100, 100]
right_pwm     = signed 16-bit integer in range [-100, 100]

Python equivalent (megajaw_uart_bridge):
struct.pack('<BBhh', 0xAA, 0x55, left_pwm, right_pwm)
```

10.2 Gripper UART Frame

The `gripper_control_node` translates `Float64MultiArray` commands to 3-byte UART frames sent to the STM32:

```
Frame format:  [0xAB][0xCD][command]

command = 0x00  → open gripper
command = 0x01  → close gripper

Serial port: /dev/ttyAMA0 @ 115200 baud (same port as motors)
```

10.3 ROS 2 Topics

Topic	Message Type	Publisher	Subscriber(s)	Purpose
/camera/image/compressed	sensor_msgs/CompressedImage	camera_driver_node / ros_gz_image	detector_node, web_interface	JPEG camera frames for

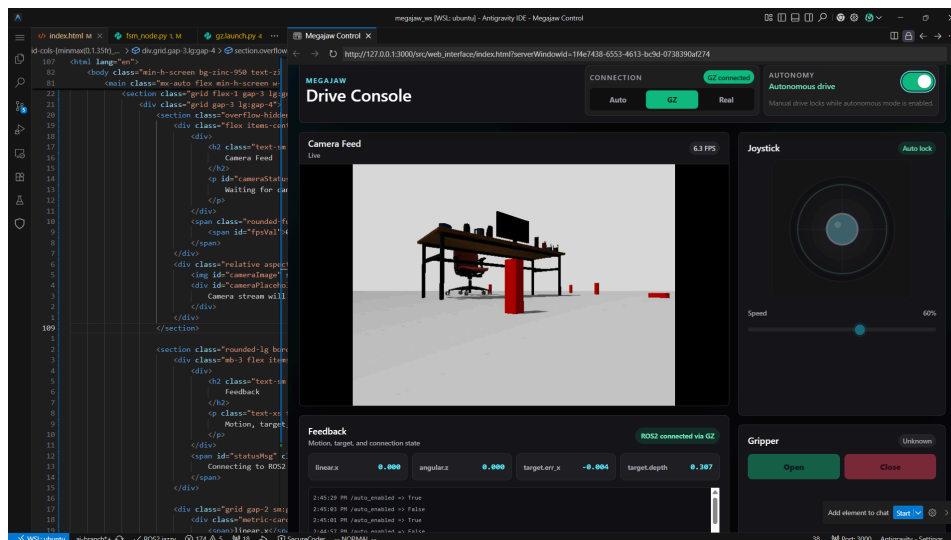
Topic	Message Type	Publisher	Subscriber(s)	Purpose
				detection and dashboard
/target_state	megajaw_interfaces/TargetControl	detector_node	fsm_node, web_interface	err_x, depth, target_detected per frame
/cmd_vel	geometry_msgs/TwistStamped	fsm_node, web_interface	diff_drive_base_controller	Linear and angular velocity commands
/gripper_controller/commands	std_msgs/Float64MultiArray	fsm_node, web_interface	gripper_controller / gripper_control_node	Finger positions: [1,-1] open, [0,0] close
/auto_enabled	std_msgs/Bool	web_interface (ROSBridge)	fsm_node	Enable/disable autonomous FSM
/joint_states	sensor_msgs/JointState	joint_state_broadcaster	robot_state_publisher	Joint positions for TF tree
/clock	roscpp_msgs/Clock	ros_gz_bridge (sim only)	All nodes (sim only)	Gazebo simulation time synchronisation

TargetControl.msg Definition

```
# megajaw_interfaces/msg/TargetControl.msg
float32 err_x          # Normalised horizontal pixel error: -(cx_detected - cx_image) / cx_image
float32 depth          # Estimated distance to target in metres (pinhole model)
bool target_detected   # True if a locked target is currently visible
```

11. Web Dashboard

The dashboard is a static HTML/JS file at `src/web_interface/index.html` that requires no build step. Open it in any modern browser and it connects to ROSBridge via ROSLIB.js on port 9090.



11.1 Connection Modes

Mode	WebSocket URL	Behaviour
Auto	localhost or zeyadcodepi.local	Tries ws://localhost:9090 first; on failure tries ws://zeyadcodepi.local:9090; cycles with 2s reconnect delay
GZ	ws://localhost:9090	Connects to local Gazebo simulation endpoint
Real	ws://zeyadcodepi.local:9090	Connects to physical robot ROSBridge server

11.2 Dashboard Features

Feature	ROS Topic	Message Type	Direction
Live camera feed	/camera/image/compressed	CompressedImage	Subscribe
Joystick manual drive	/cmd_vel	TwistStamped	Publish
Speed scaling	—	—	Client-side multiplier
Gripper open	/gripper_controller/commands	Float64MultiArray	Publish [1.0, -1.0]
Gripper close	/gripper_controller/commands	Float64MultiArray	Publish [0.0, 0.0]
Autonomous mode toggle	/auto_enabled	Bool	Publish
Target error X telemetry	/target_state	TargetControl	Subscribe
Target depth telemetry	/target_state	TargetControl	Subscribe

Feature	ROS Topic	Message Type	Direction
Connection / event log	—	—	Client-side only

When autoEnabled is true, the joystick canvas, speed slider, and gripper buttons are all disabled (is-locked CSS class applied). publishTwist() checks autoEnabled before publishing and ignores joystick input unless options.force is set (used only for the stop-on-enable transition).

12. Simulation vs Real Hardware

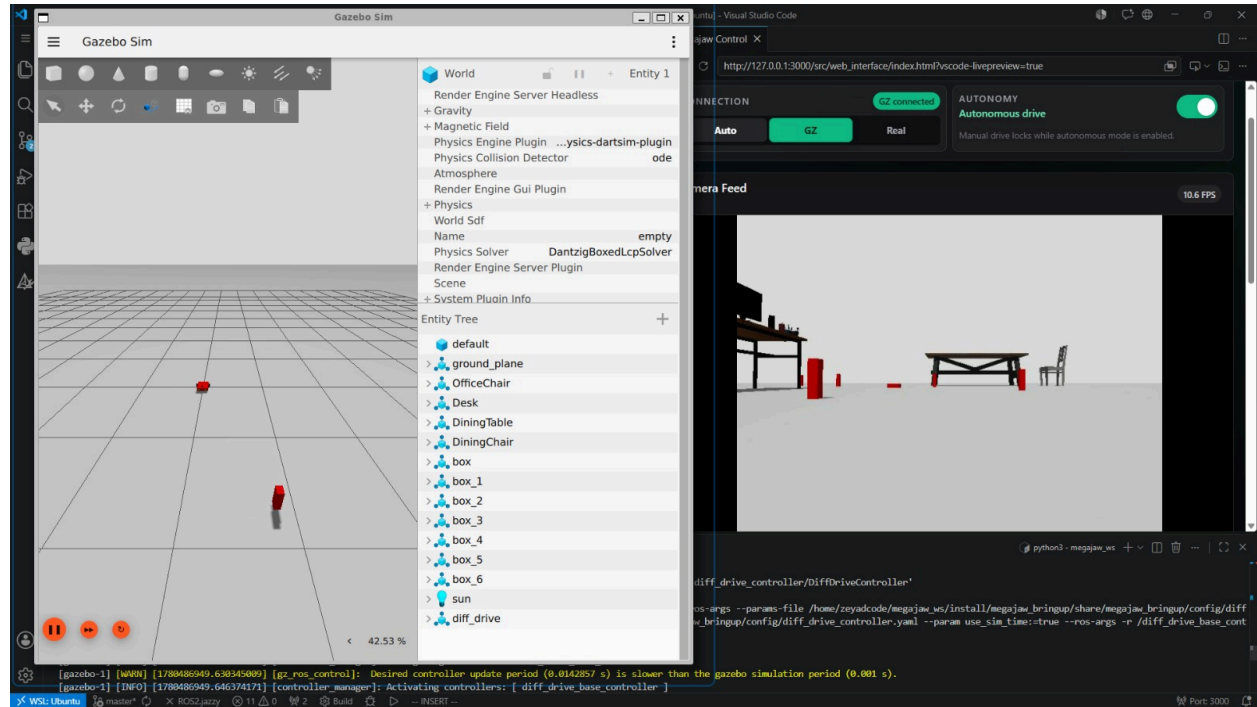


Figure 8 — Gazebo Simulation Screenshot (Replace with actual Gazebo screenshot)

Aspect	Gazebo Simulation	Real Hardware
Launch file	gz.launch.py	hardware.launch.py
backend_driver	sim	stm (default) or direct
Camera source	Gazebo camera plugin via ros_gz_image bridge	camera_driver_node polling IP Webcam HTTP stream
YOLO model	best_ncnn_model_gz (256×256)	best_ncnn_model_real (320×320)
Detection target	Red box SDF objects in modelled.sdf	Physical water bottles
Depth estimation	fx from GZ_CAM_HFOV; width only	Calibrated fx/fy; undistort + rotate; average width & height
Frame preprocessing	None	cv2.undistort() + cv2.ROTATE_90_CLOCKWISE

Aspect	Gazebo Simulation	Real Hardware
Motor control	gz_ros2_control/GazeboSimSystem	MegaJawHardware (UART) or MegaJawHardwareDirect (GPIO)
Gripper control	JointGroupPositionController via Gazebo	gripper_control_node sending 3-byte UART to STM32
Simulation time	use_sim_time: true	use_sim_time: false
close_thresh	0.05 m	0.05 m
conf_thresh	0.5 (default)	0.7 (hardware.launch.py)

13. Getting Started

13.1 Build

```
cd ~/megajaw_ws
rosdep install --from-paths src -y --ignore-src
colcon build --symlink-install
source install/setup.bash

# If Gazebo cannot resolve Fuel model assets:
export GZ_SIM_RESOURCE_PATH="$HOME/.gz/sim/models:${GZ_SIM_RESOURCE_PATH}"

# For the direct GPIO backend, start pigpiod before launching:
sudo pigpiod
```

--symlink-install is recommended during development so Python source changes in megajaw_brain take effect without a rebuild. C++ packages (megajaw_hardware, megajaw_hardware_direct) require a full rebuild after source changes.

13.2 Launch Commands

Command	Backend	Purpose
ros2 launch megajaw_bringup gz.launch.py	sim	Full Gazebo simulation: robot, world, bridge, camera, detector, FSM, ROSBridge, controllers
ros2 launch megajaw_bringup hardware.launch.py	stm (default)	Real robot: RSP, controller manager, camera driver, detector, FSM, ROSBridge, gripper node
ros2 launch megajaw_bringup display.launch.py	—	RViz2 only: URDF visualisation and TF tree inspection
ros2 launch megajaw_bringup gz.launch.py rviz:=true	sim	Gazebo simulation with RViz2
ros2 launch megajaw_bringup gz.launch.py debug:=true	sim	Simulation with detector debug window

Command	Backend	Purpose
<code>ros2 launch megajaw_bringup hardware.launch.py debug:=true</code>	stm	Real hardware with detector debug window

14. System Requirements

14.1 Software Requirements

Requirement	Version / Notes
Ubuntu	24.04 (Noble) — recommended for ROS 2 Jazzy compatibility
ROS 2	Jazzy Jalisco
Gazebo Sim	Harmonic (bundled with ros-jazzy-ros-gz-*)
ros2_control	ros-jazzy-ros2-control, diff-drive-controller, joint-state-broadcaster, position-controllers
rosbridge_server	ros-jazzy-rosbridge-suite
robot_state_publisher	ros-jazzy-robot-state-publisher
topic_tools	ros-jazzy-topic-tools (relay node for camera_info in sim)
Python	3.10+ (ROS 2 Jazzy default)
ultralytics	Latest (YOLO inference and NCNN export)
ncnn	Latest (Python bindings)
opencv-python	Latest
numpy	Latest
pyserial	Latest (for megajaw_uart_bridge)
pigpio	Latest (for megajaw_hardware_direct backend only)
colcon	python3-colcon-common-extensions
rosdep	python3-rosdep

14.2 Hardware Requirements

Requirement	Notes
Raspberry Pi	Any model running Ubuntu 24.04 with enough RAM for YOLO NCNN inference

Requirement	Notes
STM32 microcontroller	Required for stm backend; connected via /dev/ttyAMA0 at 115200 baud
H-bridge motor driver	Required for both stm and direct backends
Two DC motors	Differential drive; PWM range ± 100
Mobile phone (IP Webcam)	Streams MJPEG at <a href="http://<ip>:8080/video">http://<ip>:8080/video ; configured URLs: 192.168.1.11:8080 and 192.168.43.1:8080
pigpiod daemon	Required on RPi for direct backend; must be started before launch
Network connectivity	RPi and dev machine must share a network for ROSBridge; hostname zeyadcodepi.local used by dashboard

